AWK/GAWK

Файлы и строки

Для Unix системы существует концепция «Всё есть файл».

По умолчанию все переменные строки!!!

Необходимы операторы для обработки строк.

Операторы BASH обрабатывают строки или целочисленные значения.

Обработка текста + математика

В UNIX программы работают с:

- потоками ввода/вывода,
- перенаправить потоки ввода/вывода, операторы «>» и «<»
- конвейер/канал/пайп «|»:

```
комада1 | команда2 | команда3
```

Существуют программы фильтры, например cut, grep, sed и т.д.

Хочется:

- костомные алгоритмы для фильтрации текста,
- операции с числами с плавающей точкой,
- работа с массивами,
- подпрограммы, рекурсию,
- математику (sin, cos, log, exp),
- приемлемое быстродействие.

Наверное это Perl или Python!?



Что есть AWK?

AWK (первые буквы фамилий разработчиков) – это скриптовый язык, для потоковой обработки текста в командной строке.

Полнота по Тьюринргу.

Поддерживает регулярные выражения.

Базовый синтаксис:

```
awk options '{action}' your_file_name.txt
```

или:

```
cat your_file_name.txt | awk options '{action}'
```

или:

```
awk options -f script.awk your_file_name.txt
```

где, script.awk это:

```
{action}
```

; - разделитель между операторами.

Некоторые источники

- https://www.math.spbu.ru/user/rus/cluster/Doc/Library/awk_baluev/awk_bal_oglav.shtml
- https://pallen.asuscomm.com/ispolzovanie-awk-v-linux/
- https://routerus.com/awk-command/

Опции

-F, --field-separator - разделитель полей, используется для разбиения текста на колонки;

-f, --file - прочитать код не из стандартного вывода, а из файла;

-v, --assign - присвоить значение переменной, например foo=bar;

-b, --characters-as-bytes - считать все символы однобайтовыми;

-d, --dump-variables - вывести значения всех переменных awk по умолчанию;

-D, --debug - режим отладки, позволяет вводить команды интерактивно с клавиатуры;

-o, --pretty-print - вывести результат работы программы в файл;

-V, --version - вывести версию утилиты.

Функции-действия

print(строка) - вывод чего либо в стандартный поток вывода;

printf(строка) - форматированный вывод в стандартный поток вывода, стандарт C;

system(команда) - выполняет команду в системе;

length(строка) - возвращает длину строки;

substr(строка, старт, количество) - обрезает строку и возвращает результат;

tolower(строка) - переводит строку в нижний регистр;

toupper(строка) - переводить строку в верхний регистр.

Служебные переменные

FNR - номер обрабатываемой строки в файле;

FS - разделитель полей;

NF - количество колонок в данной строке;

NR - общее количество строк в обрабатываемом тексте;

RS - разделитель строк, по умолчанию символ новой строки;

OFS - символ разделителя полей на выводе.

$ - ссылка на колонку по номеру.

$1 – первое поле;

$2 – второе поле;

$0 – ссылается на всю строку целиком.

Строка может содержать максимально до 100 полей.

http://hepd.pnpi.spb.ru/~shevel/Book/node70.html

Шаблоны

Логика работы шаблонов подобна регулярным выражениям, ниже несколько примеров:

/^a/ - поле начинается с "a"

/a$/ - поле кончается "a"

\+   - экранирует оператор

[abc] - любой из символов "a", "b" и "c"

[a-p] - любой символ диапазона

*   - 0 или больше вхождений регулярного выражения

+   - 1 или больше вхождений регулярного выражения

?   - 0 или 1 вхождение регулярного выражения

ab|cd - "ab" или "cd"

Операции над шаблонами

Логические комбинации:

"&&", "||", "!", "()".

Принадлежность:

"~" (принадлежит) ;

"!~" (не принадлежит)

Операции отношения:

"==", "!=", ">", ">=", "<", "<=".

Простые примеры AWK

Имитация grep:

`awk '/^O/' ifile.txt` # поиск по шаблону

`awk '! /0$/'` # поиск по шаблону НЕ

`awk '/^Катя|^Игорь/ {print}'`

Сравнение без оператора IF:

`awk '$3 != $4 && $3 > 10 {print}'`

`awk '$2=="Яблоки" {print}'`

Имитация cut:

`echo '1 2 3 4' | awk '{print $3,$1}'`

Имитация wc:

`echo '1 2 3 4' | awk '{print NF}'`

`echo '1 2 3 4' | awk 'END{print NR}'`

Имитация nl:

`echo '1 2 3 4' | awk '{print NF, $0}'`

Вывести файл от строки K до строки N:

`awk 'NR==K, NR==N {print $0}' ifile.txt`

Вывести файл от строки Вася до строки Дима:

`awk '/Вася/,/Дима/ { print $1 }' ifile.txt`

Также позволяет печатать строки определенной длинны или диапазона длин и т.д.



BEGIN и END

{ } - используется для указания начала и конца блока кода.

BEGIN указывает на начало input или на те действия, которые должны быть выполнены до какого бы то ни было анализа строк.

END указывает на конец input или на те действия, которые должны быть выполнены после обработки всех строк.

Пример:

BEGIN {FS = “:”} – Устанавить разделитель полей в “:” до начала обработки строк.

END { print NR } – Печатает номер последней строки input, т.е. количество обработанных входных строк.

Переменные, присвоение

`x=1` # присвоить число

`x=abc` # присвоить значение переменной abc

`x="abc"` # присвоить строку "abc"

`x=1+2` # присвоить результат суммы

`x=$i` # присвоить значение поля i

`x=$(i+1)` # присвоить значение поля с номером i+1

`x=function(a)` # присвоить результат функции

Допускается: +=, -= , *= , /= , %=

Массив

Массивы не объявляются, а принимают значения из контекста, например:

`x[NR] = $0` # элементу массива x, индексированному NR, присваивается обрабатываемая строка.

`x[3,1] = $0` # многомерный массив

gawk supports true multidimensional arrays:

`x[2][1] = 6`

`x[“apple”]` # элементы массива могут индексироваться не числовым значением, т.е. строкой.

`delete array1` # удалить массив array1

Арифметические выражения

Возможно применить для:

Переменной;

Числа;

Строки;

Встроенной функции.

Операции:

“+”, “-“, “*”, “/”, “%”

Еще встроенные функции

`exp()` # e в степени x

`log()` # натуральный логарифм

`sqrt()` # корень квадратный

`int()` # целая часть от числа

`cos()` # косинус угла в рад.

`rand()` # случайное число от 0 до 1

`atan2(y, x)` # arctan(y/x) в радианнах.

Управляющие структуры

Условное предложение:

`if ( условие ) { предложение } [else { предложение } ]`

`if ( условие ) { предложение } [else if{ предложение } ]`

Предложения цикла:

`while ( условие ) { предложение }`

`for (выражение ; условие ; выражение ) { предложение }`

Например:

`for(i=1; i< =NF; i++)` # Аналогично циклу for в языке C”

`for (i in array)` # Цикл по элементам массива. Но, элементы массива доступны в этом случае в случайном порядке.

`break` – Немедленный выход из цикла.

`continue` – Переход к выполнению следующего предложения.

`next` – Немедленный переход к анализу следующей строки.

`exit` – Выход из программы (на конец input).

Функции

awk позволяет определять новые функции:

```
function name(parameter list) { statements }
```

Внутри тела функции можно определить локальные переменные, которые должны быть описаны в поле параметров, но отделены от них более чем одним пробелом. Например:

```
function MyFunction(i,j, a,b,c)
# a, b и c являются локальными переменными.
{
...
return m
}
```

Вызов функции:

```
MyFunction(43,61)
x=MyFunction(43,61)
```

Бонус 1: тернарный оператор

Стандартный синтаксис условия:

```
if () { ... } else { ... }
```

1 форма тернарного оператора:

```
[результат = условие? Результат 1: результат 2]
```

2 форма тернарного оператора:

```
[условие? Выражение 1: Выражение 2]
```

Пример:

```
awk '{$2=90? a++:b++ } END{print a , b}'
```

```
awk '{print ($2 == "яблоки") ? "Имя" $1: "шт." $3}'
```

Бонус 2: Код AWK в коде BASH

```
#!/bin/bash
var1=$2
ifile=$1
ofile=out.txt
cat $ifile | sort |\
   awk -v var1=$var1 '{ operator 1
   operator2 ; operato3}' > $ofile
bash_operator1
exit
```